

PwC MEAN Stack Interview Workbook — COMPLETE

4 Days × 2 Hours • 5–7 Years • Angular 18+ • Node.js • Express • MongoDB • Redis • Microservices • DSA • Azure DevOps

Built from your supplied PwC JD. This is a speaking + coding workbook: learn the answer, close the page, explain it aloud, then reproduce the code/architecture.

4-Day Roadmap

DAY	FOCUS	TARGET
1	JavaScript + Node/Express + REST/Security	Core backend + coding
2	Angular 18+ + RxJS + Signals + Forms	Frontend + reactive patterns
3	MongoDB + Redis/Auth + Microservices	Data + distributed systems
4	DSA + Azure/CI-CD + Scenarios + Mock	Interview execution

DAY 1 — JavaScript + Node.js + Express + REST + Security

Checklist

- ES6 scope/hoisting/TDZ/closure/this/prototypes
- destructuring/spread/rest/map/filter/reduce
- Promise/async-await/all/allSettled
- event loop/microtasks/macrotasks/libuv
- modules/EventEmitter/streams/buffers
- Express middleware/routing/validation/errors
- REST/status codes/pagination
- JWT/CORS/XSS/rate limiting/password hashing

JavaScript Q&A

Q. var vs let vs const?

Answer: var is function-scoped and can be redeclared. let/const are block-scoped and have a temporal dead zone before initialization. const prevents reassignment, but referenced objects can mutate.

```
if(true){let a=1;const b=2;} // a,b unavailable outside
```

Q. What is hoisting?

Answer: Declarations are processed before execution. var is initialized to undefined; let/const/class are not usable before initialization.

```
console.log(x); // undefined
var x=10;
// console.log(y); // ReferenceError
let y=20;
```

Q. What is a closure?

Answer: A function retains access to variables from its lexical outer scope after the outer function returns. Useful for private state and factories.

```
function counter(){let n=0;return()=>{++n;}}
const next=counter(); next(); //1; next(); //2
```

Q. What is lexical scope?

Answer: Variable lookup depends on where code is written, not where the function is called.

```
const x="outer"; function f(){return x;}
```

Q. Explain this.

Answer: For normal functions, this depends on the call site. Arrow functions capture this lexically. bind/call/apply can control this for normal functions.

```
const u={name:"A",hi(){return this.name}};
u.hi(); // A
const h=u.hi.bind(u); h(); // A
```

Q. What is prototypal inheritance?

Answer: Objects delegate property lookup through a prototype chain. JavaScript classes are built on prototypes.

```
const animal={speak(){return "sound"}};
const dog=Object.create(animal); dog.speak();
```

Q. map/filter/reduce?

Answer: map transforms; filter selects; reduce accumulates.

```
[1,2,3].map(x=>x*2);
[1,2,3].filter(x=>x>1);
[1,2,3].reduce((s,x)=>s+x,0);
```

Q. Spread vs rest?

Answer: Spread expands values; rest collects remaining values.

```
const b=[...a,3];
function sum(...nums){return nums.reduce((a,b)=>a+b,0)}
```

Q. What is destructuring?

Answer: It extracts object properties or array values into variables.

```
const {name,age}=user;
const [first,second]=items;
```

Q. Shallow vs deep copy?

Answer: Shallow copies top-level values but nested references are shared. Deep copy separates nested data using a suitable cloning strategy.

```
const a={n:{x:1}}; const b={...a};
b.n.x=9; // a.n.x also changes
```

Q. Promise vs async/await?

Answer: Promise is the asynchronous abstraction; async/await is syntax for consuming Promises more readably and does not block the event loop.

```
async function load(){try{return await api()}catch(e){throw e}}
```

Q. Promise.all vs allSettled?

Answer: all rejects when one rejects. allSettled waits for every operation and returns each outcome.

```
await Promise.all([a(),b()]);
await Promise.allSettled([a(),b()]);
```

Q. Debounce vs throttle?

Answer: Debounce waits until calls stop; throttle limits frequency. Search commonly uses debounce; scroll/resize often uses throttle.

```
search$.pipe(debounceTime(300));
```

Q. What is immutability?

Answer: Avoiding direct mutation makes state changes predictable and works well with reactive state/change detection.

```
const next={...state,count:state.count+1};
```

Node.js + Express Q&A

Q. Why Node.js for I/O-heavy applications?

Answer: Its event-driven non-blocking I/O model lets the JS thread handle other work while I/O waits. CPU-heavy work may need workers/processes.

```
const data=await fs.promises.readFile("file.txt","utf8");
```

Q. What is libuv?

Answer: A Node dependency providing the event loop and asynchronous I/O abstractions, including a worker pool for selected operations.

Q. What is the event loop?

Answer: It coordinates the call stack with asynchronous callbacks so Node can continue processing instead of blocking on I/O.

```
console.log("A");
setTimeout(()=>console.log("timer"),0);
Promise.resolve().then(()=>console.log("micro"));
console.log("B");
```

Q. What is Express middleware?

Answer: A request-pipeline function that can inspect/modify req/res, end the response or call next().

```
app.use((req,res,next)=>{console.log(req.method);next();});
```

Q. How implement centralized errors?

Answer: Handlers pass errors to next(error); final middleware maps known errors to safe responses and logs unexpected failures.

```
app.use((err,req,res,next)=>{
  console.error(err);
  res.status(err.statusCode||500).json({message:"Request failed"});
});
```

Q. How validate API input?

Answer: Validate body/query/params against a schema before business logic and return consistent 4xx errors.

```
const data=schema.parse(req.body);
```

Q. PUT vs PATCH?

Answer: PUT generally replaces a resource representation; PATCH applies partial changes.

```
PUT /users/42
PATCH /users/42
```

Q. 401 vs 403?

Answer: 401 means authentication is missing/invalid; 403 means authenticated but not permitted.

Q. What is CORS?

Answer: A browser security mechanism controlling cross-origin requests. Configure trusted origins, methods and headers explicitly.

```
app.use(cors({origin:["https://app.example.com"]}));
```

Q. How paginate an API?

Answer: Offset/limit is simple; cursor/keyset pagination scales better for large/changing data when backed by a stable index.

```
db.orders.find({createdAt:{$lt:cursor}})
.sort({createdAt:-1}).limit(20);
```

Q. What is graceful shutdown?

Answer: Stop new traffic, finish/time out active requests, close DB/broker connections and exit cleanly.

```
process.on("SIGTERM",async()=>{
  server.close(); await db.close(); process.exit(0);
});
```

Q. How avoid blocking Node?

Answer: Avoid synchronous heavy filesystem/crypto/CPU work in request handlers. Use async APIs, workers or separate services.

REST + Security Q&A

Q. What makes an API RESTful?

Answer: Resource-oriented URLs, standard HTTP methods/status codes, stateless requests and representations. REST is an architectural style.

```
GET /orders/42
POST /orders
PATCH /orders/42
DELETE /orders/42
```

Q. How secure an Express API?

Answer: HTTPS, authentication, authorization, validation, secure headers, rate limiting, controlled CORS, password hashing, secret management and safe logging.

```
app.use(helmet());
app.use(rateLimit({windowMs:60000,max:100}));
```

Q. How store passwords?

Answer: Never plaintext. Use Argon2 or bcrypt with appropriate parameters and unique salts.

```
const hash=await bcrypt.hash(password,12);
const ok=await bcrypt.compare(password,hash);
```

Q. What is XSS?

Answer: Cross-site scripting executes attacker-controlled content in a browser. Mitigate with output encoding, safe DOM APIs, CSP and careful sanitization.

Q. What is rate limiting?

Answer: Limit requests per client/key over a time window to reduce abuse and protect resources.

Q. What is API idempotency?

Answer: Repeated requests with the same idempotency key should create one intended business effect; critical for payments/orders.

```
POST /payments
Idempotency-Key: abc-123
```

Day 1 coding

```
function twoSum(nums,target){
const seen=new Map();
for(let i=0;i<nums.length;i++){
const need=target-nums[i];
if(seen.has(need))return[seen.get(need),i];
seen.set(nums[i],i);
}
return [];
}
// O(n) time, O(n) space
```

DAY 2 — Angular 18+ + RxJS + Signals + Forms + Routing

Checklist

- Standalone/component + module architecture
- DI/services
- signal/computed/effect
- Observable/Subject/BehaviorSubject
- switchMap/mergeMap/concatMap/exhaustMap
- async pipe/lifecycle
- Reactive Forms/FormArray/validators
- config-driven forms
- routing/lazy loading/params/AuthGuard
- HTTP interceptor/JWT
- OnPush/change detection/list tracking/performance

Angular Q&A

Q. What is Angular component architecture?

Answer: Components contain template, styles and logic. Modern Angular supports standalone components; enterprise applications may still use NgModule/module architecture.

```
@Component({selector: "app-user", standalone: true,
template: `<h2>{{name()}}</h2>`}
export class User {name=signal("Alice");}
```

Q. What is dependency injection?

Answer: Angular's injector supplies dependencies, improving reuse, testing and separation of concerns.

```
constructor(private userService:UserService){}
```

Q. What are Signals?

Answer: signal stores writable state; computed derives values; effect reacts for side effects.

```
count=signal(0);
double=computed(()=>this.count()*2);
increment(){this.count.update(v=>v+1);}
```

Q. Signals vs RxJS?

Answer: Signals suit synchronous state and derived UI values. RxJS suits asynchronous streams, events and complex stream composition. They can interoperate.

```
results=toSignal(this.api.search(this.query$),{initialValue:[]});
```

Q. Observable vs Promise?

Answer: Promise represents one eventual result. Observable can emit multiple values and supports RxJS operators/subscription lifecycle.

```
this.http.get("/api/users").subscribe(users=>this.users=users);
```

Q. Subject vs BehaviorSubject?

Answer: Subject does not replay current state. BehaviorSubject requires an initial value and replays the latest value.

```
const state$=new BehaviorSubject({loading:false});
state$.next({loading:true});
```

Q. switchMap vs mergeMap?

Answer: switchMap keeps the latest inner stream; mergeMap allows concurrent inner streams.

```
search$.pipe(debounceTime(300),switchMap(q=>api.search(q)));
```

Q. concatMap vs exhaustMap?

Answer: concatMap queues work sequentially. exhaustMap ignores new emissions while current work runs; useful for submit prevention.

```
saveClicks$.pipe(exhaustMap(()=>api.save()));
jobs$.pipe(concatMap(j=>api.process(j)));
```

Q. How prevent RxJS memory leaks?

Answer: Prefer async pipe. For manual subscriptions use lifecycle-aware teardown such as takeUntilDestroyed.

```
this.api.users$.pipe(takeUntilDestroyed(this.destroyRef)).subscribe();
```

Q. What is an HTTP interceptor?

Answer: Central request/response pipeline for auth headers, correlation IDs, loading and common errors.

```
const r=req.clone({setHeaders:{Authorization:`Bearer ${token}`}});
return next(r);
```

Q. What is an AuthGuard?

Answer: A route navigation check. It is not a substitute for backend authorization.

```
export const guard={()=>inject(Auth).loggedIn()?true:
inject(Router).createUrlTree(['/login'])};
```

Q. Reactive vs template-driven forms?

Answer: Reactive forms are model-driven and explicit, better for complex/dynamic forms and testing. Template-driven is simpler for small forms.

```
form=new FormGroup({email:new FormControl("",[Validators.required,Validators.email])});
```

Q. setValue vs patchValue?

Answer: setValue expects the full structure; patchValue updates only supplied fields.

```
form.patchValue({email:"a@x.com"});
```

Q. What is FormArray?

Answer: A reactive structure for a dynamic list of controls/groups.

```
skills=new FormArray([new FormControl("Angular"),new FormControl("Node")]);
```

Q. What is a config-driven form?

Answer: Metadata defines fields, types, defaults and validation; a reusable builder/renderer creates the form.

```
[{key:"name",type:"text",required:true},
{key:"age",type:"number"}]
```

Q. What is lazy loading?

Answer: Load feature code only when needed to reduce initial bundle/startup work.

```
{path:"orders",loadComponent:()=>import("./orders").then(m=>m.OrdersComponent)}
```

Q. How optimize Angular?

Answer: Lazy load, reduce unnecessary change detection, track lists, avoid duplicate requests, use async pipe/signals, optimize bundles/images and measure.

```
changeDetection:ChangeDetectionStrategy.OnPush
```

Q. What is role-based UI?

Answer: Show/disable features by role for UX; backend independently enforces authorization.

```
canEdit=computed(()=>auth.roles().includes("ADMIN"));
```

Q. What is change detection?

Answer: Angular checks whether bindings need updating. Performance improves by reducing unnecessary checks and choosing suitable reactive/change-detection patterns.

Q. Why track/trackBy lists?

Answer: Stable identity lets Angular reuse DOM nodes instead of recreating them unnecessarily.

```
@for(item of items; track item.id){<div>{{item.name}}</div>}
```

Day 2 RxJS drill

```
// Latest search wins
search$.pipe(
  debounceTime(300),
  distinctUntilChanged(),
  switchMap(q=>api.search(q)),
  catchError(()=>of([]))
);

// Ignore repeated submits until current save completes
submit$.pipe(exhaustMap(()=>api.save()));
```

DAY 3 — MongoDB + Redis + JWT/OAuth + Microservices + Kafka

Checklist

- embed/reference
- indexes/compound/explain/ESR
- match/group/project/sort/lookup/unwind/facet
- large data/cursor pagination
- transactions/consistency
- Redis cache-aside/TTL/invalidation/failure
- JWT access/refresh/OAuth/RBAC
- API Gateway/service discovery
- Kafka/RabbitMQ
- retry/backoff/idempotency/DLQ
- circuit breaker/Saga/eventual consistency
- logs/metrics/tracing

MongoDB Q&A

Q. Embedding vs referencing?

Answer: Embed small bounded data normally read together. Reference large/shared/independently updated/unbounded relationships.

```
{_id:1,address:{city:"Kolkata"}} // embedded
{_id:1,customerId:ObjectId("...")} // reference
```

Q. What is an index?

Answer: A structure that accelerates lookup/sort but costs storage and write/update overhead.

```
db.users.createIndex({email:1},{unique:true});
```

Q. How diagnose a slow query?

Answer: Use explain/profile data, inspect winning plan and keys/docs examined, verify index/query shape, optimize and measure again.

```
db.orders.find({customerId:id}).sort({createdAt:-1}).explain("executionStats");
```

Q. What is a compound index?

Answer: An index over multiple fields; field order matters for useful query/sort patterns.

```
db.orders.createIndex({customerId:1,createdAt:-1});
```

Q. What is ESR?

Answer: Equality → Sort → Range is a useful starting guideline for compound indexes; validate against real workload/explain.

```
db.orders.createIndex({customerId:1,status:1,createdAt:-1});
```

Q. \$match vs \$group?

Answer: \$match filters documents; \$group groups documents and calculates accumulations.

```
db.orders.aggregate([
  {$match:{status:"PAID"}},
  {$group:{_id:"$customerId",total:{$sum:"$amount"}}}
]);
```

Q. \$lookup?

Answer: Combines related documents from another collection in an aggregation pipeline.

```
{$lookup:{from:"users",localField:"userId",foreignField:"_id",as:"user"}}
```

Q. \$unwind?

Answer: Deconstructs an array so each element becomes a pipeline document.

```
{$unwind:"$items"}
```

Q. \$facet?

Answer: Runs multiple pipelines over the same input, useful for data plus count/statistics.

```
{$facet:{data:[{$skip:0},{limit:20}],total:[{$count:"count"}]}}
```

Q. How handle millions of documents?

Answer: Selective indexes, projection, cursor pagination, efficient aggregation, suitable archival/partitioning and measurement.

```
db.orders.find({createdAt:{lt:cursor}}).sort({createdAt:-1}).limit(50);
```

Q. When use transactions?

Answer: When multiple writes must satisfy an atomic consistency requirement. Prefer good modeling before adding transactions.

```
await session.withTransaction(async()=>{
  await orders.insertOne(o,{session});
});
```

```
await inventory.updateOne(f,u,{session});
});
```

Q. What is data consistency?

Answer: Maintain business invariants and choose an appropriate consistency model; distributed systems may require eventual consistency.

Redis + Authentication Q&A

Q. What is cache-aside?

Answer: Check Redis first; on miss read DB and populate Redis, usually with TTL. DB remains source of truth.

```
const v=await redis.get(key);
if(v)return JSON.parse(v);
const d=await db.users.findOne({id});
await redis.set(key,JSON.stringify(d),{EX:300});
```

Q. What is cache invalidation?

Answer: Delete/update cache after source changes; TTL/versioned keys can add safety.

```
await db.users.updateOne({id},{set:update});
await redis.del(`user:${id}`);
```

Q. What if Redis goes down?

Answer: If Redis is only a cache, fall back safely to DB with timeouts/circuit breaking and monitor database load.

```
try{return await cache.get(k)}
catch{return await db.users.findOne({id})}
```

Q. Authentication vs authorization?

Answer: Authentication verifies identity; authorization decides permission.

Q. Explain JWT.

Answer: A signed token contains claims. Server validates signature and relevant claims such as expiry, issuer and audience.

```
const payload=jwt.verify(token,publicKey,{issuer:"api",audience:"web"});
```

Q. Why access + refresh tokens?

Answer: Short-lived access tokens reduce exposure. Refresh tokens obtain new access tokens and need strong protection/lifecycle controls.

```
POST /auth/refresh
```

Q. OAuth vs JWT?

Answer: OAuth is an authorization framework; JWT is a token format. OAuth can use JWTs but they are not equivalent.

Q. What is RBAC?

Answer: Permissions are grouped into roles and users receive roles; backend services enforce authorization.

```
if(!user.roles.includes("ADMIN"))return res.sendStatus(403);
```

Microservices + Messaging Q&A

Q. Why microservices?

Answer: Independent business capabilities can deploy/scale separately. Trade-offs include operational complexity, network failures and distributed consistency.

Q. What is an API Gateway?

Answer: A client-facing entry point for routing and cross-cutting policies such as auth integration and rate limiting.

Q. REST vs messaging?

Answer: REST is usually synchronous request/response. Messaging enables asynchronous decoupling, buffering and event-driven workflows.

```
POST /orders
OrderCreated -> Kafka -> Inventory/Notify/Analytics
```

Q. Kafka core concepts?

Answer: Producer, topic, partition, offset, broker and consumer group. Partitions provide parallelism; groups distribute partition processing.

```
producer.send({topic:"orders",messages:[{key:orderId,value:JSON.stringify(event)}}]);
```

Q. What is idempotency?

Answer: Repeated processing of the same logical operation produces one intended business effect.

```
if(await db.processedEvents.findOne({eventId}))return;
await process(event); await db.processedEvents.insertOne({eventId});
```

Q. What is a DLQ?

Answer: A dead-letter queue stores repeatedly failing messages for investigation/replay without blocking normal traffic.

Q. What is a circuit breaker?

Answer: After repeated downstream failures, calls temporarily stop; a later probe tests recovery and prevents cascading failures.

Q. What is Saga?

Answer: A distributed workflow of local transactions with compensating actions when later steps fail.


```
OrderCreated -> ReserveInventory -> Payment
PaymentFailed -> ReleaseInventory
```

Q. How handle downstream failure?

Answer: Timeouts, bounded exponential-backoff retries, circuit breakers, idempotency, safe fallbacks and asynchronous queues where appropriate.

Q. What is eventual consistency?

Answer: Services may temporarily differ after asynchronous events but converge as processing completes.

Q. What is service discovery?

Answer: A mechanism for services to locate healthy instances dynamically instead of hard-coded addresses.

Q. What is distributed tracing?

Answer: Trace/correlation context is propagated across services to follow latency and failures end-to-end.

Q. Kafka vs RabbitMQ?

Answer: Kafka is strong for durable event streams, partitions, replay and high-throughput pipelines. RabbitMQ is strong for queue/routing/task patterns. Choose based on delivery, ordering, replay and workload needs.

Architecture Drill

```
Angular
|
HTTPS + JWT
v
API Gateway
+-- User Service ---- MongoDB
+-- Order Service --- MongoDB --- Kafka
+-- Payment Service - MongoDB
|
Inventory/Notify
Redis = cache
Logs + Metrics + Traces = observability
CI/CD = build -> test -> scan -> deploy
```

DAY 4 — DSA + Azure DevOps + Production Scenarios + Mock Checklist

- Two Sum
- first unique
- anagram
- sliding window
- valid parentheses
- binary search
- reverse linked list
- Big-O
- Azure Repos/Pipelines/Boards/Artifacts
- CI/CD/secrets/rollback
- Docker
- mock interview

DSA Q&A + Solutions

Q. Two Sum

Answer: Map value→index; check target-x before storing current. O(n) time/O(n) space.

```
function twoSum(a,t){
  const m=new Map();
  for(let i=0;i<a.length;i++){const n=t-a[i];
    if(m.has(n))return[m.get(n),i];m.set(a[i],i);}
  return [];
```

Q. First non-repeating character

Answer: Count with Map, then scan for first count 1. O(n).

```
function firstUnique(s){
  const m=new Map();
  for(const c of s)m.set(c,(m.get(c)||0)+1);
  for(const c of s)if(m.get(c)===1)return c;
  return null;
}
```

Q. Anagram

Answer: Compare character frequencies. O(n) time for a normal finite character set.

```
function anagram(a,b){
  if(a.length!==b.length)return false;const m=new Map();
  for(const c of a)m.set(c,(m.get(c)||0)+1);
  for(const c of b){if(!m.has(c))return false;m.set(c,m.get(c)-1);}
  return [...m.values()].every(v=>v===0);
}
```

Q. Maximum sum of K consecutive values

Answer: Sliding window: calculate first K, then subtract outgoing/add incoming. O(n).

```
function maxK(a,k){
  let s=0;for(let i=0;i<k;i++)s+=a[i];let best=s;
  for(let i=k;i<a.length;i++){s+=a[i]-a[i-k];best=Math.max(best,s);}
  return best;
}
```

Q. Valid parentheses

Answer: Stack opening brackets; every closing bracket must match top. O(n).

```
function valid(s){
  const st=[],p={'(':')','[':']','{':'}';
  for(const c of s){if(p[c].includes(c))st.push(c);
    else if(st.pop()!==p[c])return false;}
  return st.length===0;
}
```

Q. Binary search

Answer: On sorted input, halve the search range each iteration. O(log n).

```
function bs(a,t){
  let l=0,r=a.length-1;
  while(l<=r){const m=Math.floor((l+r)/2);
    if(a[m]===t)return m;if(a[m]<t)l=m+1;else r=m-1;}
  return -1;
}
```

Q. Reverse linked list

Answer: Keep prev/current, reverse next pointer and advance. O(n) time/O(1) extra space.

```
function reverse(head){
  let p=null,c=head;
  while(c){const n=c.next;c.next=p;p=c;c=n;}
  return p;
}
```

Q. What is Big-O?

Answer: Asymptotic growth of time/space with input size, ignoring constants and lower-order terms.

Q. Array lookup vs Map lookup?

Answer: Array index access is typically $O(1)$; Map lookup is average $O(1)$; unsorted value search is $O(n)$.

Q. When use a stack?

Answer: Matching brackets, nested structures, DFS, undo and monotonic-stack patterns.

Q. When use a queue?

Answer: BFS, FIFO processing and task workflows.

Q. When use sliding window?

Answer: Contiguous subarray/substring problems where the answer can be updated as the window moves.

Q. What say before coding?

Answer: Clarify input/output and edge cases, give brute force, optimize, explain complexity, code, then test.

Azure / DevOps Q&A

Q. What is CI/CD?

Answer: CI integrates changes with automated validation. CD automates delivery/deployment through controlled environments.

```
Build -> Test -> Scan -> Package -> Dev -> Test -> Approval -> Prod
```

Q. Azure Repos/Pipelines/Boards/Artifacts?

Answer: Repos stores source; Pipelines automates build/test/deploy; Boards manages work; Artifacts stores packages/build outputs.

Q. How manage environment configuration?

Answer: Keep config outside source where possible; use secure secret/config stores and never commit credentials.

```
API_URL = environment config
JWT_SECRET = secret-store reference
```

Q. How rollback?

Answer: Use versioned immutable artifacts and keep a known-good version. Roll back quickly, verify health, then investigate.

```
deploy myapp:1.42
# incident
deploy myapp:1.41
```

Q. Why Docker?

Answer: Packages the application/runtime into a repeatable image, reducing environment drift.

```
FROM node:22-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
CMD ["node","server.js"]
```

Q. What should CI pipeline include?

Answer: Install dependencies, lint, unit tests, build, security/dependency checks, package an immutable artifact and deploy through controlled environments.

Senior Production Scenario Q&A

Q. API latency suddenly increases.

Answer: Measure endpoint latency, traces, DB queries, cache hit rate, CPU/memory and downstream latency. Compare baseline, mitigate immediate impact, fix root cause and add prevention.

Q. MongoDB query becomes slow.

Answer: Use explain/profile, inspect indexes/query shape, reduce payload, optimize aggregation/indexing, benchmark and monitor.

Q. Angular dashboard makes duplicate calls.

Answer: Inspect lifecycle/subscriptions/network waterfall; centralize shared data; cache/share or use state/signals; remove duplicate subscriptions.

Q. Kafka event processed twice.

Answer: Assume at-least-once delivery; make consumer idempotent with event ID/unique constraint/processed-event record; use retries and DLQ.

Q. Payment succeeds but client times out and retries.

Answer: Use an idempotency key; persist operation state/result and return the existing result for repeated keys.

Q. Redis has stale data.

Answer: Define invalidation ownership; delete/update on writes; use TTL/versioned keys and monitor.

Q. One microservice is down.

Answer: Isolate failure with timeout/circuit breaker/fallback; queue non-critical work; return graceful partial results when safe.

Q. Traffic suddenly increases 10x.

Answer: Cache read-heavy data, scale stateless services, optimize DB/indexes/replicas, queue async work, rate-limit and load-test.

Q. Bad deployment reaches production.

Answer: Rollback to known-good artifact; inspect logs/traces/config/schema changes; fix through pipeline and add regression test.

Q. How monitor microservices?

Answer: Structured logs with correlation IDs, metrics for latency/error/throughput, distributed tracing, dashboards and meaningful alerts/SLOs.

Q. Design order→payment→inventory.

Answer: Use bounded services with local persistence. Synchronous calls only where immediate result is required; events for decoupled work; idempotency and Saga/compensation for distributed workflow; observability everywhere.

Q. How handle a slow downstream service?

Answer: Set bounded timeouts; use circuit breaking, fallback/cache or asynchronous processing based on business requirements. Never wait indefinitely.

Final 40-Minute Mock

- 90-second self introduction
- Explain current project architecture
- Explain exact personal responsibility
- Signals vs RxJS with real examples
- switchMap/mergeMap/concatMap/exhaustMap
- JWT access/refresh + RBAC
- Secure Express API
- Design users/orders MongoDB schema + indexes
- Aggregation: match + lookup + unwind + group
- Redis use case + Redis failure plan
- Order → payment → inventory design
- REST vs Kafka
- Duplicate Kafka event
- Circuit breaker + retry/backoff + DLQ + Saga
- Solve Two Sum + sliding window aloud
- Explain CI/CD + Azure DevOps
- Production incident + root cause
- Performance optimization with measurable impact
- Difficult team situation using STAR
- Why PwC / why this role

Senior Answer Framework

Coding: clarify → brute force → optimize → code → test → complexity. Architecture: requirements → APIs → data → communication → failure handling → security → scaling → observability → deployment. Behavioral: Situation → Task → Action → Result → learning.

Final Night-Before Checklist

- Project architecture from memory
- Three STAR stories ready
- Production bug/root-cause story ready
- Performance improvement with measurable result ready
- Signals/RxJS/operators clear
- JWT/RBAC clear
- MongoDB indexes/aggregation clear
- Redis cache/failure clear
- Kafka/idempotency/DLQ/Saga/circuit breaker clear
- At least 4 DSA patterns can be coded
- Azure/CI-CD clear

- Do not bluff unknown technologies; state assumptions
- Think aloud while coding
- Ask clarifying questions before design
- Keep first answer concise and expand if asked

Interview Morning — 20 Minutes

- 5 min — project architecture + responsibilities
- 5 min — Angular/RxJS/Signals
- 5 min — Node/Mongo/Redis/JWT
- 5 min — DSA patterns + Big-O

Final reminder: for a 5–7 year role, do not answer only with definitions. Connect the concept to an implementation, trade-off, failure mode and monitoring. If you have not used a technology, be honest and explain how you would investigate it safely.